

# COMP1844 Information Analysis and Visualisation

## Coursework Report

**Module:** COMP1844 Information Analysis and Visualisation

**Student:** Trịnh Tuấn Phong

**Student ID:** GCS220282

**Lecturer:** Le Tran Ngoc Tran

**Submission Date:** 25/04/2004

## Table of Contents

|                                                |   |
|------------------------------------------------|---|
| I. Introduction .....                          | 2 |
| II. Single-Line Transport Graph.....           | 2 |
| 2.1 Data Collection .....                      | 2 |
| 2.2 Python Code.....                           | 2 |
| 2.3 Generated Visualization .....              | 4 |
| III. Multi-Line Connected Transport Graph..... | 4 |
| 3.1 Python Code.....                           | 4 |
| 3.2 Generated Visualization .....              | 7 |
| IV. Network Analysis .....                     | 8 |
| 4.1 Python Code.....                           | 8 |
| 4.2 Results and Interpretation .....           | 9 |
| V. Conclusion.....                             | 9 |
| VI. References.....                            | 9 |
| VII. Appendix .....                            | 9 |

## I. Introduction

This coursework uses Python to create and analyse part of the London Underground network. A graph data structure is used where stations are represented as nodes and railway connections are represented as edges. The project contains three tasks. Task 1 creates a small transport network with three lines. Task 2 expands the network into a larger connected graph with five lines. Task 3 calculates several statistics from the completed network. The program only uses NumPy, pandas, NetworkX and Matplotlib as required.

## II. Three-Lines Transport Graph

### 2.1 Data Collection

The station names were selected from the London Underground network. The distances between stations were estimated using Google Maps and are shown in kilometres. Three Underground lines were included: Bakerloo, Piccadilly and Northern. Different colours were used to make each line easy to identify.

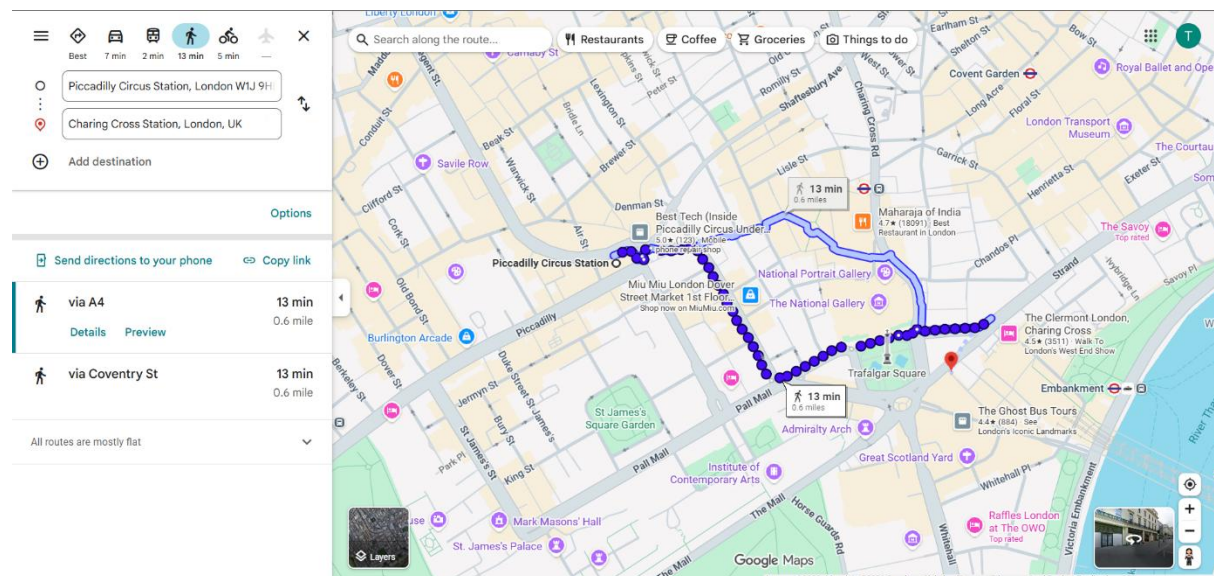


Figure 1. The distances between stations are shown in kilometres of Google Maps

### 2.2 Python Code

Task 1 begins by creating an empty graph using **NetworkX**. Five stations are added and connected with edges. Each edge stores two pieces of information: the distance between stations and the colour of the Underground line. The node positions are manually arranged so the graph looks similar to a transport map. Finally, Matplotlib is used to draw the graph, display station names, edge labels and a legend.

Step 1: Import Tools – Loads necessary Python libraries to build graph structures and plot images.

Step 2: Define Connections – Creates an empty network graph and adds station pairs along with their distances and line colors.

Step 3: Set Positions – Sets 2D map coordinates for each station so they align neatly.

Step 4: Draw Stations and Tracks – Draws stations as gray dots, tracks as colored lines, and displays station names and distance values.

Step 5: Add Map Legend and Display – Adds a color key showing which line each color represents, cleans up axis lines, and displays the final plot

```
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
```

Figure 2. The Python program begins by importing the required libraries.

```
G1 = nx.Graph()

edges_t1 = [
    ('Oxford Circus', 'Piccadilly Circus', 0.5, 'saddlebrown'),
    ('Piccadilly Circus', 'Charing Cross', 0.6, 'saddlebrown'),
    ('Charing Cross', 'Leicester Square', 0.3, 'black'),
    ('Piccadilly Circus', 'Leicester Square', 0.4, 'blue'),
    ('Leicester Square', 'Covent Garden', 0.2, 'blue')
]

for u, v, w, c in edges_t1:
    G1.add_edge(u, v, weight=w, color=c)

pos_t1 = {
    'Oxford Circus': (1.0, 4.0),
    'Piccadilly Circus': (2.0, 2.3),
    'Charing Cross': (3.0, 0.5),
    'Leicester Square': (3.0, 2.3),
    'Covent Garden': (4.0, 3.0)
}

plt.figure(figsize=(9, 7))
colors1 = [G1[u][v]['color'] for u, v in G1.edges()]

nx.draw_networkx_nodes(G1, pos_t1, node_color='lightgray', node_size=700)
nx.draw_networkx_edges(G1, pos_t1, edge_color=colors1, width=4)

label_offsets = {
    'Oxford Circus': (1.45, 4.0),
    'Piccadilly Circus': (1.55, 2.1),
    'Charing Cross': (3.0, 0.15),
    'Leicester Square': (3.5, 2.15),
    'Covent Garden': (4.5, 3.0)
}
nx.draw_networkx_labels(G1, label_offsets, font_size=11)

edge_labels1 = nx.get_edge_attributes(G1, 'weight')
nx.draw_networkx_edge_labels(G1, pos_t1, edge_labels={k: f"{v} km" for k, v in
edge_labels1.items()}, font_size=10)

for label, color in [('Piccadilly', 'blue'), ('Northern', 'black'), ('Bakerloo',
'saddlebrown')]:
    plt.plot([], [], color=color, label=label, linewidth=3)

plt.legend(title="Key", loc='lower right', bbox_to_anchor=(0.95, 0.05), borderpad=1.5,
edgecolor='black', fontsize=10)
```

```
plt.title("Task 1: Initial Public Transport Network Section of London", fontsize=14,
fontweight='bold', pad=20)
plt.xlim(0.5, 5.5)
plt.ylim(-0.2, 4.5)
plt.axis('off')
plt.tight_layout()
plt.show()
```

Figure 3. Python code for three lines graph

### 2.3 Generated Visualization

The output shows a simple transport network with five stations connected by three Underground lines. Every connection displays its approximate distance in kilometres. Different colours make it easy to distinguish between the Bakerloo, Piccadilly and Northern lines. The graph is clear and satisfies the requirements of Task 1.

#### Task 1: Initial Public Transport Network Section of London

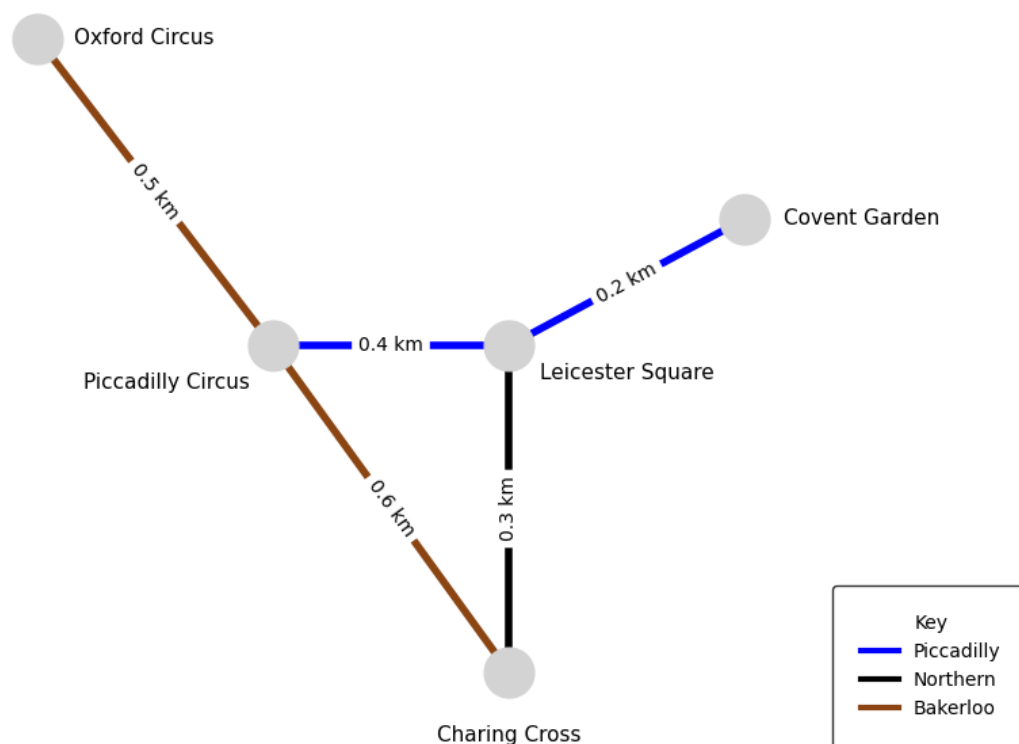


Figure 4. Output of task 1 code

## III. Multi-Line Connected Transport Graph

### 3.1 Python Code

The program extends the graph by adding additional nodes and edges for the new transport lines. Each edge contains the transport line name and the distance between two stations. Colours are stored for each line and used when drawing the graph. The completed graph is

then displayed with labelled stations, coloured lines, distance labels, and a key showing the name of each transport line.

Step 1: Organize Line Data – Groups stations into a structured list containing each line's name, color, and consecutive station distances.

Step 2: Build Full Network – Creates a new graph and loops through the line list to automatically connect neighboring stations along each route.

Step 3: Map Layout Setup – Defines precise coordinates for placing each station dot and its text label on the screen.

Step 4: Identify Transfer Stations – Counts how many lines pass through each station to separate interchange stations (serving multiple lines) from standard stations.

Step 5: Render Map Features – Draws standard stations as gray circles and interchange stations as white circles with black outlines. It then draws colored track lines, distance labels, a legend box, and displays the map.

```
import matplotlib.pyplot as plt
import networkx as nx

network_data = [
    ('Bakerloo', 'saddlebrown', [
        ("Regent's Park", 0.7), ('Oxford Circus', 0.5), ('Piccadilly Circus', 0.6),
        ('Charing Cross', 0.2), ('Embankment', 0.5), ('Waterloo', 0.0)
    ]),
    ('Piccadilly', 'blue', [
        ('Green Park', 0.4), ('Piccadilly Circus', 0.4), ('Leicester Square', 0.2),
        ('Covent Garden', 0.4), ('Holborn', 0.6), ('Russell Square', 0.0)
    ]),
    ('Northern', 'black', [
        ('Warren Street', 0.3), ('Goodge Street', 0.4), ('Tottenham Court Road', 0.3),
        ('Leicester Square', 0.3), ('Charing Cross', 0.2), ('Embankment', 0.0)
    ]),
    ('Central', 'red', [
        ('Bond Street', 0.2), ('Oxford Circus', 0.4), ('Tottenham Court Road', 0.4),
        ('Holborn', 0.4), ('Chancery Lane', 0.7), ('St. Paul's', 0.0)
    ]),
    ('Victoria', 'darkturquoise', [
        ('Victoria', 0.9), ('Green Park', 0.7), ('Oxford Circus', 0.9),
        ('Warren Street', 0.4), ('Euston', 0.6), ('King's Cross', 0.0)
    ])
]

G2 = nx.Graph()
for line, color, stations in network_data:
    for i in range(len(stations) - 1):
        G2.add_edge(stations[i][0], stations[i+1][0], weight=stations[i][1], color=color)

G2['Charing Cross']['Embankment']['color'] = 'saddlebrown'

pos_t2 = {
    'Oxford Circus': (1.0, 4.0),
    'Piccadilly Circus': (2.0, 2.3),
    'Charing Cross': (3.0, 0.5),
    'Leicester Square': (3.0, 2.3),
```

```

'Regent\'s Park': (1.0, 6.0),
'Embankment': (3.0, -0.5),
'Waterloo': (4.4, -1.0),
'Green Park': (0.2, 2.3),

'Tottenham Court Road': (3.0, 4.0),
'Goodge Street': (3.0, 5.2),
'Warren Street': (3.0, 6.2),
'Euston': (3.75, 7.0),
'King\'s Cross': (5.4, 7.0),

'Covent Garden': (3.4, 3.0),
'Holborn': (4.0, 4.0),
'Russell Square': (5.0, 5.5),
'Chancery Lane': (5.1, 4.0),
'St. Paul\'s': (6.8, 4.0),

'Bond Street': (0.0, 4.0),
'Victoria': (0.2, 0.1)
}

label_offsets_t2 = {
'Oxford Circus': (0.5, 4.25),
'Piccadilly Circus': (1.6, 2.05),
'Charing Cross': (2.4, 0.5),
'Leicester Square': (3.6, 2.05),
'Regent\'s Park': (0.4, 6.0),
'Embankment': (3.55, -0.35),
'Waterloo': (4.85, -1.0),
'Green Park': (-0.3, 2.3),

'Goodge Street': (3.6, 5.2),
'Warren Street': (2.4, 6.2),
'Euston': (3.75, 7.3),
'King\'s Cross': (5.4, 7.3),
'Tottenham Court Road': (2.2, 3.8),

'Covent Garden': (4.0, 3.0),
'Holborn': (4.2, 3.7),
'Russell Square': (5.65, 5.5),
'Chancery Lane': (5.1, 4.3),
'St. Paul\'s': (6.8, 4.3),

'Bond Street': (-0.5, 3.8),
'Victoria': (0.6, 0.1)
}

plt.figure(figsize=(16, 12))
colors2 = [G2[u][v]['color'] for u, v in G2.edges()]

station_counts = {}
for line, color, stations in network_data:
    for station, _ in stations:
        station_counts[station] = station_counts.get(station, 0) + 1

interchange_stations = [station for station, count in station_counts.items() if count > 1]
regular_stations = [node for node in G2.nodes() if node not in interchange_stations]

nx.draw_networkx_nodes(G2, pos_t2, nodelist=regular_stations, node_color='lightgray',
node_size=600)

```

```

nx.draw_networkx_nodes(G2, pos_t2, nodelist=interchange_stations, node_color='white',
edgecolors='black', linewidths=2.5, node_size=600)

nx.draw_networkx_edges(G2, pos_t2, edge_color=colors2, width=4)
nx.draw_networkx_labels(G2, label_offsets_t2, font_size=10.5, font_weight='bold')

edge_labels2 = nx.get_edge_attributes(G2, 'weight')
nx.draw_networkx_edge_labels(G2, pos_t2, edge_labels={k: f"{v} km" for k, v in
edge_labels2.items() if v > 0}, font_size=10.5)

for line, color, _ in network_data:
    plt.plot([], [], color=color, label=line, linewidth=4)
plt.legend(title="Line Legend Map", loc='lower right', borderpad=1.5, edgecolor='black',
fontsize=11)

plt.title("Task 2: Expanded and Connected London Transport Network Section (Refined)",
fontsize=14, fontweight='bold', pad=20)
plt.xlim(-1.2, 10.2)
plt.ylim(-1.5, 8.5)
plt.axis('off')
plt.tight_layout()
plt.show()

```

Figure 5. Python code for multi line transport graph

### 3.2 Generated Visualization

The completed graph represents a larger section of the London Underground. Five coloured lines, multiple interchange stations and distance labels are displayed clearly. The layout is organised and easy to read, allowing users to follow different routes across the connected network.



Figure 6. Map tube for task 2

## Task 2: Expanded and Connected London Transport Network Section (Refined)

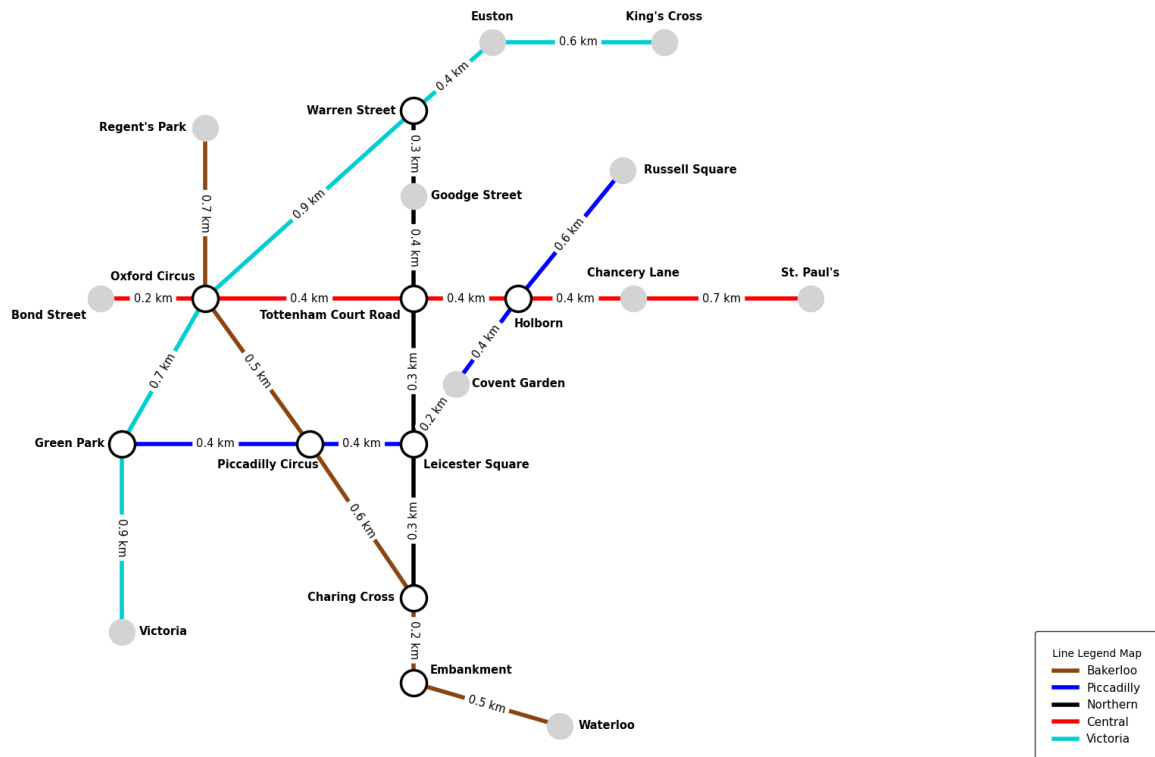


Figure 7. Output of task 2 code

## IV. Network Analysis

### 4.1 Python Code

The program reads all edge distances from the graph and stores them in a list. NumPy functions are then used to calculate the required statistics. Finally, the results are printed to the console.

```
distances = [d['weight'] for u, v, d in G2.edges(data=True) if d['weight'] > 0]

df_metrics = pd.DataFrame({
    'Network Metric': [
        'Total length of the transport network',
        'The average distance between the stations',
        'The standard deviation of the distances'
    ],
    'Value (km)': [
        round(np.sum(distances), 2),
        round(np.mean(distances), 2),
        round(np.std(distances), 2)
    ]
})

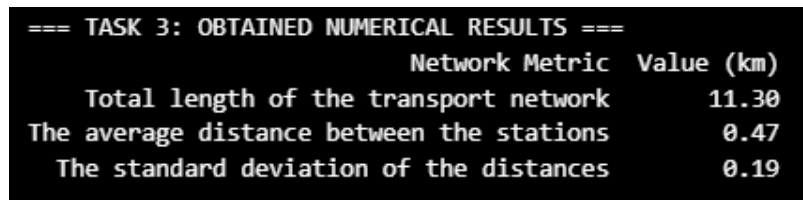
print("=== TASK 3: OBTAINED NUMERICAL RESULTS ===")
print(df_metrics.to_string(index=False))
```



Figure 8. Python code program for calculating and computing

## 4.2 Results and Interpretation

The calculated results show the overall characteristics of the transport network. The total length represents the sum of all connections, while the average distance shows the typical distance between two neighbouring stations. The standard deviation indicates how much the distances vary across the network. These values help evaluate whether the transport network is balanced and realistic.



| Network Metric                            | Value (km) |
|-------------------------------------------|------------|
| Total length of the transport network     | 11.30      |
| The average distance between the stations | 0.47       |
| The standard deviation of the distances   | 0.19       |

Figure 9. The computing's result

## V. Conclusion

This coursework successfully developed a transport network using graph data structures in Python. The first task created a simple graph with three transport lines and distance labels. The second task expanded the network to include five connected lines with a consistent colour scheme and interchange stations. Finally, the third task analysed the network by calculating its total length, average distance, and standard deviation. Overall, the project demonstrates how graph visualisation can be used to represent and analyse a transport network in a clear and effective way.

## VI. References

- Google Maps. (2025). *Google Maps*. <https://maps.google.com>
- NetworkX Developers. (2025). *NetworkX Documentation*. <https://networkx.org>
- Matplotlib Development Team. (2025). *Matplotlib Documentation*. <https://matplotlib.org>
- NumPy Developers. (2025). *NumPy Documentation*. <https://numpy.org>
- pandas Development Team. (2025). *pandas Documentation*. <https://pandas.pydata.org>

## VII. Appendix

<https://github.com/phongttgcs220282/COMP1844-Coursework>

## Declaration of AI Use

Please append this page to the end of your assignment when you have used AI during the process of undertaking the assignment to acknowledge the ways in which you have used it.

I have used AI while undertaking my assignment in the following ways:

- To develop research questions on the topic – YES
- To create an outline of the topic –NO
- To explain concepts – YES
- To support my use of language – YES
- To summarise the following articles/resources:
  1. I used AI to help formulate and refine research questions for my topic.
  2. I utilized AI to explain complex concepts encountered during the project.
  3. I used AI as an assistance tool to improve and support my use of language throughout the report.